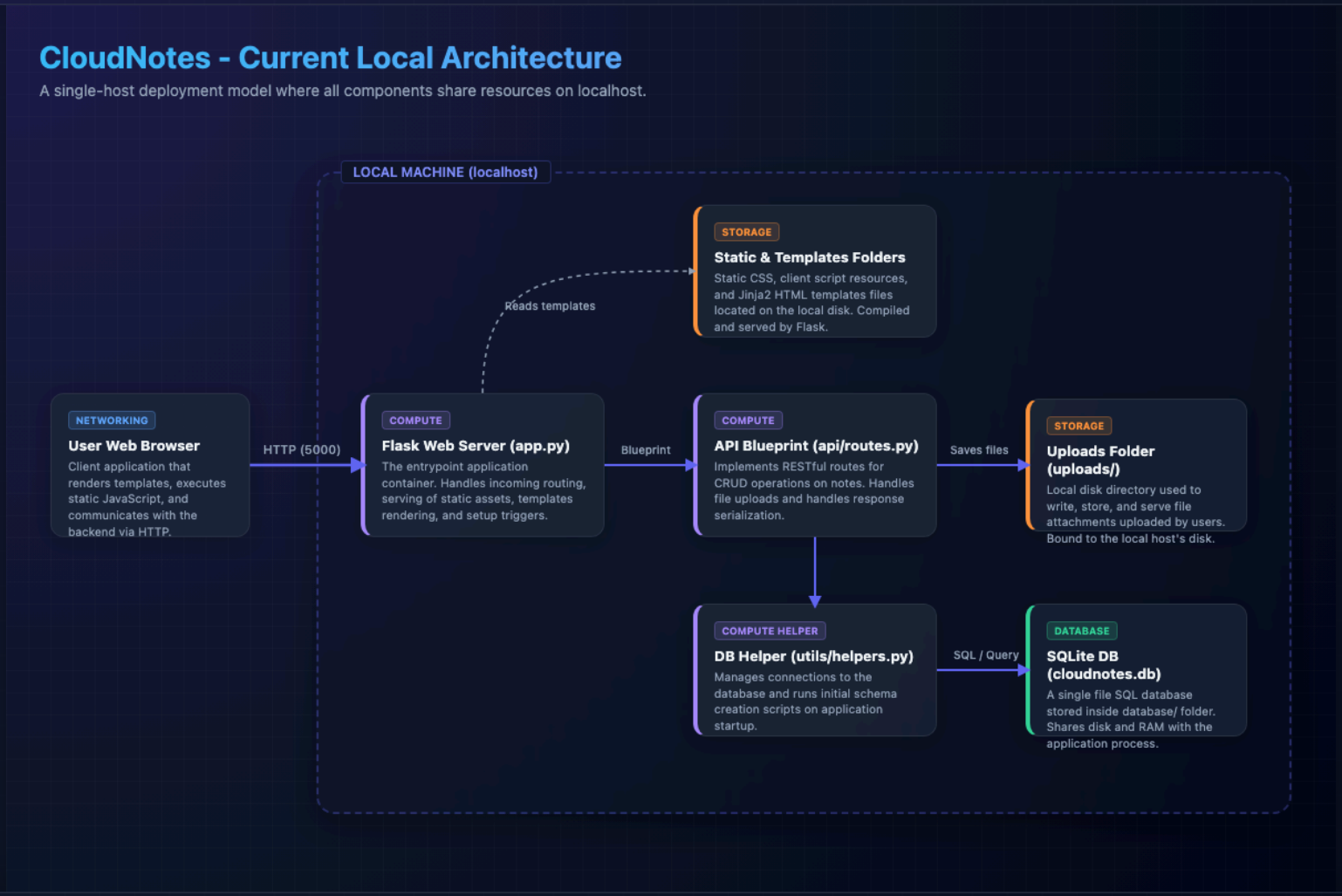
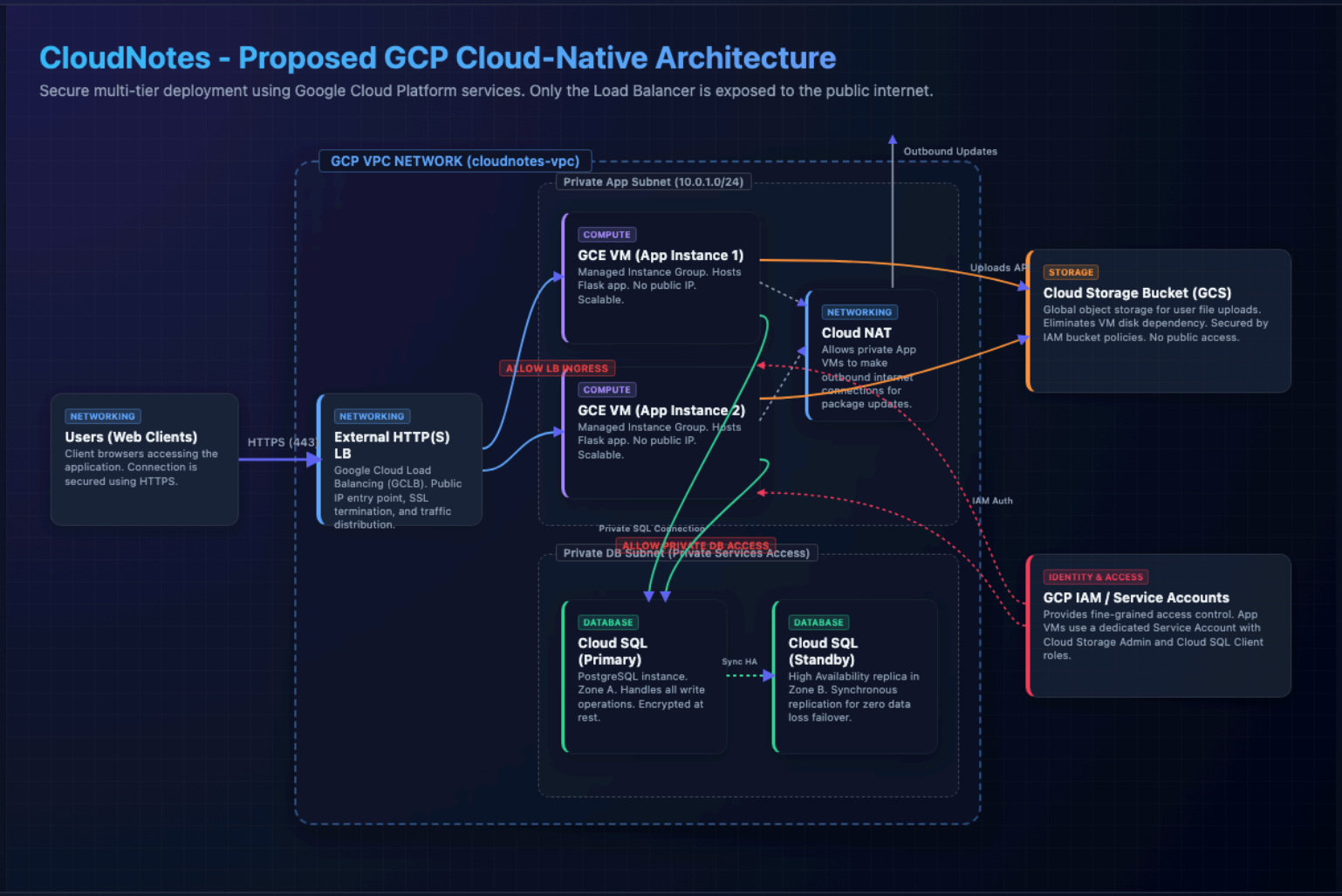
01. Current Local Architecture Diagram



ARCHITECTURAL REASONING

The current local architecture runs on a single workstation, coupling the Flask application process, templates, static files, and uploads to the local filesystem and memory. While simple, SQLite database access locks writes during concurrent actions and storing files in a local uploads folder prevents horizontal scaling across multiple nodes. This baseline highlights the operational constraints that necessitate moving CloudNotes to a decoupled, cloud-native architecture.

02. Proposed Cloud Architecture Diagram



ARCHITECTURAL REASONING

The proposed GCP architecture introduces an external HTTP(S) load balancer as the single entry point, keeping the compute and database layers protected in private subnets. GCE Managed Instance Groups host the application workload and scale dynamically, while all local file dependencies are removed by migrating file uploads to a secure Google Cloud Storage bucket. Cloud SQL PostgreSQL is configured with synchronous multi-zone replication to Zone B, ensuring zero data loss and automated failover.

03. Component-to-Service Mapping Table

Component-to-Service Mapping Table

Mapping local application components to enterprise-grade, cloud-native services on Google Cloud Platform.

COMPONENT	SERVICE CATEGORY	PROPOSED GCP SERVICE	JUSTIFICATION
Flask App Server app.py & routes.py	COMPUTE	**Compute Engine (GCE) VM** Managed Instance Group (MIG)	Hosts Flask application workloads on virtual machines located in a private VPC subnet. Enables auto-scaling and automatic VM healing to match active traffic demand.
SQLite Database cloudnotes.db	DATABASE	**Cloud SQL for PostgreSQL** Multi-Zone HA Instance	Replaces the file-bound SQLite database with a fully-managed relational database service. Provides synchronous multi-zone replication, automated point-in-time recovery backups, and scalability.
Uploads Folder uploads/	STORAGE	**Google Cloud Storage (GCS)** Standard Regional Bucket	Decouples user-uploaded attachments (images, PDFs) from the local VM file systems. Establishes stateless VMs by moving persistence to an infinitely-scalable, durable object store.
Static Assets static/	STORAGE	**Cloud Storage + Cloud CDN** Static bucket cached globally	Offloads static assets (CSS, JS, images) from application servers. Storing them in a bucket and caching them on edge points via CDN reduces page load latency and saves compute cycles.
Local Loopback Network localhost:5000	NETWORKING	**External Application Load Balancer** Google Cloud Load Balancing (GCLB)	Acts as the single public entry point for secure user requests. Translates external HTTPS (443) traffic and loads balances connections across healthy private app instances.
Host Isolation & Filesystem Local System Rights	SECURITY	**VPC Firewall & Cloud IAM** Identity & Access Policies	Ensures zero trust. VPC firewalls block direct public internet access to app VMs and DB. Dedicated Service Accounts grant granular, role-based access for VM-to-Storage API calls.

ARCHITECTURAL REASONING

This mapping table establishes the direct translation of local CloudNotes components into native, fully managed GCP services. Compute workloads are decoupled onto auto-scaling VMs, persistent uploads are offloaded to high-durability object storage, and the local SQLite file is replaced with a scalable PostgreSQL database. This migration strategy ensures stateless compute instances, multi-zone data durability, and network isolation via VPC firewalls and IAM permissions.

04. Distributed Systems Risk Analysis

Distributed Systems Risk Analysis

Failure mode evaluation and infrastructure response metrics for CloudNotes in a production cloud environment.

SCENARIO A

Database Zone Outage

The physical availability zone (AZ) hosting the primary relational database suffers a critical power or cooling loss.

WHAT HAPPENS

Without high availability, active database connections are broken, and the application becomes completely unavailable, generating **500 Internal Server Errors** for all write/read actions.

CAP THEOREM ANALYSIS

Under the CAP Theorem, during a network partition or zone failure, we must choose between Consistency (C) and Availability (A). For transactional note data, we choose **Consistency (CP)**. Active writes fail briefly while ensuring no conflicting states or lost commits occur.

MITIGATION / RESOLUTION

Deploy Cloud SQL in a **High Availability (HA)** configuration with a synchronous standby replica in Zone B. GCP automatically redirects connections to the replica via DNS failover within 60 seconds of outage detection.

SCENARIO B

10x Traffic Surge

A viral event or campaign creates a sudden 10x traffic spike on the public application endpoints.

WHY SINGLE VM FAILS

A single VM has hard constraints on physical CPU, memory, socket descriptors, and WSGI worker threads. Once saturated, requests queue up, latency increases exponentially, and the OS drops connection packets, leading to **502 Bad Gateway / 504 Timeout** errors.

SCALE MECHANICS

The bottlenecks of SQLite (lock contention on write) and localized disk uploads compound this, as multiple threads cannot write concurrently, causing the local filesystem to thrash.

MITIGATION / RESOLUTION

Decouple application state by using GCS and Cloud SQL. Place the GCE instances inside a **Managed Instance Group (MIG)** with an Autoscaling policy (e.g. scale on 70% CPU) paired with an External HTTP(S) Load Balancer.

SCENARIO C (OPTIONAL)

Storage Drive Failure / State Loss

The host virtual machine's storage disk fails, or a scaled-down event destroys ephemeral local instance disks.

WHY LOCAL STORAGE FAILS

Hosting uploads in a local `uploads/` directory couples data directly to a single VM. If the instance is auto-recreated or scaled down by autoscaling, all user attachment files are lost forever.

STATE INCONSISTENCY

Furthermore, in a multi-instance scaling setup, users hitting VM-1 cannot access files uploaded to VM-2, resulting in broken file links (404 errors) across different user sessions.

MITIGATION / RESOLUTION

Migrate all file attachments from local disks to **Google Cloud Storage (GCS)**. GCS buckets offer 11 9s of durability, store data redundantly, and act as a shared media repository accessible globally by all VMs.

ARCHITECTURAL REASONING

The distributed systems risk analysis evaluates the behavior of CloudNotes under infrastructure failure and rapid scaling events. Implementing a multi-zone Cloud SQL deployment resolves database zone failure (Scenario A) by providing auto-failover, while accepting a temporary recovery window in line with CP consistency under the CAP theorem. For traffic spikes (Scenario B), a Google Cloud Load Balancer working with auto-scaling compute pools distributes incoming load, preventing single-node bottlenecks.

05. Architecture Decision Record (ADR)

Architecture Decision Records (ADR)

Systematic log of the critical cloud architectural designs, constraints, and engineering trade-offs.

ADR-001 • DATABASE

Migrate SQLite to Cloud SQL PostgreSQL

Database Tier

CONTEXT

SQLite couples database states to a single VM local disk, locking write access and blocking application scaling and failover redundancy.

ALTERNATIVES CONSIDERED

SQLite on shared persistent disk (rejected due to multi-writer block locks); self-hosted PostgreSQL (rejected due to administration overhead).

TRADE-OFFS

- ✓ **Managed HA Failover:** Sync Zone B replica guarantees 99.95% uptime and automated failover.
- ✗ **Increased Cost:** Relational Database Service charges higher operational costs than local file storage.

ADR-002 • STORAGE

Move File Uploads to Google Cloud Storage

Storage Tier

CONTEXT

Flask writes uploaded files to local `uploads/` directory. Dynamic scaling and instance replacements destroy data and break user paths.

ALTERNATIVES CONSIDERED

Shared Network File System / Filestore NFS (rejected: high base cost); local disk background sync scripts (rejected: replication latency/instability).

TRADE-OFFS

- ✓ **Stateless Compute:** Enables server nodes to scale up/down freely with 11 9s of object storage durability.
- ✗ **Code Refactoring:** Demands integrating the GCP Storage SDK client libraries into Flask upload helper routines.

ADR-003 • COMPUTE

Compute Engine Managed Instance Groups

Compute Tier

CONTEXT

Need a reliable compute fabric for Python WSGI workloads that handles spikes, provides health checks, and integrates with private subnets.

ALTERNATIVES CONSIDERED

Cloud Run Serverless (rejected to demonstrate classic 3-tier networking); Kubernetes/GKE cluster (rejected: complex orchestration overhead).

TRADE-OFFS

- ✓ **Network Isolation:** Traditional VPC subnets lock VMs in private networks behind firewall ingress rules.
- ✗ **Infrastructure Management:** Requires creating instance templates, custom startup scripts, and OS image configurations.

ARCHITECTURAL REASONING

The Architecture Decision Records document the engineering justifications and trade-offs behind our core design choices. We prioritize managed high availability, durable object storage, and network isolation, accepting the higher operational costs of Cloud SQL and GCS over local files. By formalizing these decisions in ADRs, we align development patterns with enterprise-scale cloud operations and ensure clean migration paths.